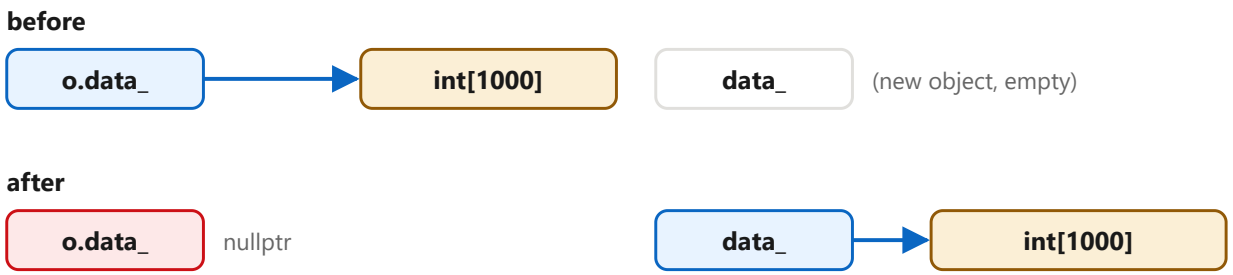
Writing Move Operations

How to write the move constructor and move assignment of a class that owns a resource, why they must be `noexcept`, and why the best answer is often to write nothing. Assumes Copy versus Move.

The move constructor, step by step

A class that owns a raw resource needs its **own** move constructor, because the compiler-generated one would copy the pointer and leave two owners. Steal the pointer and reset the source, so the source destructor frees nothing.



The buffer was never touched: only two pointers changed hands.

Steal, then reset: two steps per member.

```
class Buffer {
    int* data_; std::size_t n_;
public:
    explicit Buffer(std::size_t n) : data_(new int[n]{}), n_(n) {}
    ~Buffer() { delete[] data_; }
    // keep the example short: no copying
    Buffer(const Buffer&) = delete;

    // steal and reset, one line each
    Buffer(Buffer&& o) noexcept
        : data_(std::exchange(o.data_, nullptr)), n_(std::exchange(o.n_, 0))
{}
};
```

- `std::exchange(x, v)` returns the old `x` and sets it to `v`: exactly "steal and reset".
- After the move, `o.data_` is `nullptr`, and `delete[] nullptr` is harmless, so the source can be destroyed safely.
- A move constructor takes `T&&` (not `const`: it must change the source) and should be `noexcept`.

Move assignment and copy-and-swap

Move assignment must also release what the target already owns. The simplest correct version reuses the move constructor through **copy-and-swap**: take the parameter by value, then swap.

```
// by value: an lvalue argument is COPIED, an rvalue is MOVED
Buffer& operator=(Buffer o) noexcept {
    std::swap(data_, o.data_);              // take the new contents ...
    // ... and hand ours to o, whose destructor frees them
    std::swap(n_, o.n_);
    return *this;
}
```

STYLE	GOOD	COST
copy-and-swap (by value)	one function, handles self-assignment, strong guarantee	an extra move or copy even when storage could be reused
`operator=(const T&)` plus `operator=(T&&)`	can reuse existing storage, fastest	two functions to write and keep correct

- For a move assignment written by hand, check for self-assignment (`if (this ≠ &o)`), free your own resource, then steal.
- If you do not own a raw resource, write none of this (see below).

Why moves must be `noexcept`

`std::vector` promises that if growth throws, the vector is unchanged (the **strong guarantee**). Moving elements into the new buffer could throw halfway, so the vector uses `std::move_if_noexcept`: it moves only if the move is `noexcept`, and copies otherwise.

move marked noexcept
growth: move every element

move NOT noexcept
growth: COPY every element

Why: if a move threw halfway, the old buffer would be damaged and the strong guarantee lost.

Copies keep the old buffer intact, so the vector falls back to copying.

Compiled: 8 push_backs of a type whose move is not noexcept made 7 extra copies.

noexcept decides whether growth is a round of moves or a round of copies.

ELEMENT TYPE	COPIES DURING 8 PUSH_BACKS	MOVES
move constructor **not** noexcept	7	8
move constructor `noexcept`	0	15

```
// vector moves on growth
struct Fast { Fast(Fast&&) noexcept { /* ... */ } /* ... */ };
// vector COPIES on growth
struct Slow { Slow(Slow&&)          { /* ... */ } /* ... */ };
```

RULE Mark move constructors and move assignments `noexcept` whenever they cannot throw. Moves that only swap or reset pointers never throw.

Rule of Zero first

The best move operations are the ones you do not write. If every member already knows how to copy and move itself, the compiler-generated operations are correct.

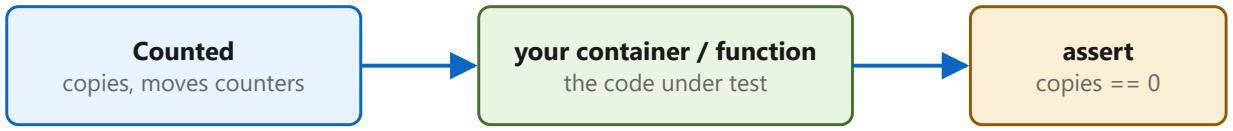
RULE	YOU WRITE	WHEN
Rule of Zero	nothing	members are `vector`, `string`, `unique_ptr` and so on
Rule of Five	destructor, copy and move constructor, copy and move assignment	the class owns a raw resource itself

```
// Rule of Zero: the compiler's copy and move are right
class Document {
    std::string title_;
    std::vector<int> pages_;
    // makes the class move-only automatically
    std::unique_ptr<Cache> cache_;
};
```

- Declare any of the five and the compiler stops generating some of the others: a user-declared destructor or copy operation suppresses the implicit moves, so a move silently becomes a copy.
- To keep the defaults after declaring something, write `= default` explicitly.

Testing your move operations

You cannot see a move in the output, so test for it. A tiny **counting type** records every copy and move, and a test asserts the numbers you expect.



Count operations in a test, then assert the numbers.

```cpp
struct Counted {
    static inline int copies = 0, moves = 0;
    Counted() = default;
    Counted(const Counted&)     { ++copies; }
    Counted(Counted&&) noexcept { ++moves;  }
};

std::vector<Counted> v;
v.reserve(8);
v.push_back(Counted());          // moves == 1, copies == 0
```

CHECK	WHAT IT PROVES
`copies == 0` after building a vector of temporaries	the code moves instead of copying
`std::is_nothrow_move_constructible_v<T>` with `static_assert`	the move is really `noexcept`, so `vector` will use it
a moved-from object can be destroyed and assigned	the moved-from state keeps the class invariant

```cpp
// fails to compile if you forget noexcept
static_assert(std::is_nothrow_move_constructible_v<Buffer>);
static_assert(std::is_nothrow_move_assignable_v<Buffer>);
```

INVARIANTS A moved-from object must still be safe to destroy and to assign to. If your class promises "never empty", a move that empties it breaks the promise: reset to a valid state, or document that moved-from objects are special.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
The move constructor, step by step	Programming: Principles and Practice	17.4 to 17.5 Copying and moving; essential operations	501 to 512
	C++ Move Semantics	6.3 Dealing with broken invariants	115 to 124
Move assignment and copy-and-swap	Programming: Principles and Practice	17.6 to 17.9 Other operations, remaining Vector problems, changing size	513 to 523
Why moves must be noexcept	Effective Modern C++	Item 14, Declare functions noexcept if they won't emit exceptions	108 to 115